Sharing Shiny Applications Reproducibly with Docker

A practical walkthrough of Dockerizing a Shiny app so collaborators can run it on any machine

Ronald 'Ryy' G. Thomas

2025-06-28

Table of contents

1 Introduction **2**
- 1.1 Motivations . . . 3
- 1.2 Objectives . . . 3

2 Prerequisites and Setup **4**

3 What is Docker for Shiny? **4**

4 Getting Started: The Naive Approach **5**
- 4.1 The Application . . . 5
- 4.2 Sending the App to Joe . . . 7
- 4.3 Error 1: Shiny Not Installed . . . 7
- 4.4 Error 2: System Libraries Missing . . . 7
- 4.5 Error 3: Package DT Not Found . . . 7
- 4.6 Error 4: palmerpenguins Not Found . . . 7
- 4.7 Error 5: Port Already in Use . . . 7
- 4.8 Error 6: Version Incompatibility . . . 8

5 The Docker Approach **8**
- 5.1 The Dockerfile . . . 8
- 5.2 Building the Image . . . 9
- 5.3 Sharing the Image . . . 9
- 5.4 Running the Container . . . 10
- 5.5 Alternative: Single-File Approach . . . 10
- 5.6 Things to Watch Out For . . . 11
- 5.7 Lessons Learnt . . . 12
  - 5.7.1 Conceptual Understanding . . . 12
  - 5.7.2 Technical Skills . . . 12
  - 5.7.3 Gotchas and Pitfalls . . . 13
- 5.8 Limitations . . . 13

5.9 Opportunities for Improvement 13

6 Wrapping Up **14**

7 See Also **14**

8 Reproducibility **15**

9 Let's Connect **15**

Figure 1: A shipping container yard representing the concept of containerised, portable software environments.

Just as shipping containers standardised global freight, Docker containers standardise software delivery – including interactive Shiny applications.

1 Introduction

I did not really know how unreliable it was to share a Shiny application as a collection of R scripts until my colleague – let us call him Joe – spent an entire afternoon installing system libraries, R packages, and Shiny Server before he could even launch the application. The app ran perfectly on my machine. Joe's Linux Mint workstation disagreed.

The problem with Shiny is that it compounds the standard reproducibility challenge. An R Markdown document requires R, packages, and possibly LaTeX. A Shiny application requires all of that plus a running web server, reactive dependencies, and often JavaScript libraries or database connections. Each additional dependency is another opportunity for something to break on a collaborator's machine.

This post walks through the complete workflow for Dockerizing a Shiny application: writing the Dockerfile, building the image, sharing it with a collaborator, and running the app in a container. The approach eliminates every environment mismatch by packaging the entire runtime into a single portable image.

More formally, this post is the Shiny-specific case study of the Container layer of the Workflow Construct described in post 52. The container-layer fundamentals (Dockerfile authoring, base image selection, dependency installation, port binding) are documented in post 32; this post addresses the additional concerns that arise specifically when the container serves an interactive Shiny application: process supervision (Shiny Server vs. ShinyProxy), TCP port exposure, persistent file storage for user uploads, and reverse-proxy authentication.

1.1 Motivations

- I had sent a Shiny application to a colleague as a zip file of R scripts, and he spent hours resolving package version conflicts and missing system libraries before the app would launch.
- The Shiny ecosystem introduces dependencies beyond standard R analysis: `httpuv`, `htmlwidgets`, `shiny` itself, and any JavaScript libraries used by UI components all need to be present and compatible.
- I wanted a method for sharing interactive applications that would work regardless of the recipient's operating system, R version, or installed packages.
- My previous post on Dockerizing R Markdown convinced me that the same approach could work for Shiny, but the server component introduces additional considerations.
- The experience taught me that "it works on my machine" is particularly dangerous for Shiny apps, where the failure mode is a blank browser tab with no useful error message.

1.2 Objectives

1. Demonstrate the failure modes of sharing Shiny applications without Docker by describing the typical sequence of errors a collaborator encounters.
2. Write a complete Dockerfile that packages a Shiny application with all its dependencies, including R packages, system libraries, and data files.
3. Build, push, and pull a Docker image so that a collaborator can run the Shiny app with a single `docker run` command.
4. Document the port mapping, volume mounting, and container lifecycle for interactive Shiny applications.

I am documenting my learning process here. If you spot errors or have better approaches, please let me know.

The application begins on one machine, but collaborators need to run it on theirs.

2 Prerequisites and Setup

Before proceeding, you will need:

- **Docker Desktop** installed on your machine (macOS, Windows, or Linux)
- **R 4.0+** and the `shiny` package installed locally (for development)
- **A Docker Hub account** (free) for pushing and sharing images
- Basic familiarity with Shiny application structure (`ui.R`/`server.R` or single-file `app.R`)

If you have not used Docker before, the essential concept is this: a Docker image is a snapshot of a complete computing environment, and a Docker container is a running instance of that image. When you share an image, the recipient gets an exact copy of your environment, including the running Shiny server.

3 What is Docker for Shiny?

Docker packages an application and its entire computing environment into a single portable unit. For Shiny applications specifically, the image contains R, all required packages, the Shiny server, system libraries, data files, and the application code itself.

Think of it like shipping a complete web server instead of just the application files. The naive approach to sharing a Shiny app is like sending someone your restaurant recipe and hoping they have the same kitchen, ingredients, and equipment. Docker is like shipping the entire kitchen, pre-stocked and ready to cook.

The key difference between Dockerizing Shiny and Dockerizing R Markdown is the network layer. An Rmd analysis produces a static output file. A Shiny app runs a persistent web server that listens on a port. The Docker container must expose that port so the host machine's browser can connect to the running application.

4 Getting Started: The Naive Approach

To understand why Docker matters for Shiny, consider what happens when you share an application without it.

4.1 The Application

Assume you have a single-file Shiny app (`app.R`) that displays an interactive scatter plot of the Palmer Penguins dataset. The app uses `shiny` for the framework, `ggplot2` for plotting, `DT` for interactive tables, and `palmerpenguins` for data.

```r
library(shiny)
library(ggplot2)
library(DT)
library(palmerpenguins)

ui <- fluidPage(
  titlePanel("Palmer Penguins Explorer"),
  sidebarLayout(
    sidebarPanel(
      selectInput(
        "species", "Species:",
        choices = c(
          "All",
          unique(
            as.character(penguins$species)
          )
        )
      ),
      selectInput(
        "x_var", "X Variable:",
        choices = c(
          "bill_length_mm",
          "bill_depth_mm",
          "flipper_length_mm",
          "body_mass_g"
        )
      ),
      selectInput(
        "y_var", "Y Variable:",
        choices = c(
          "flipper_length_mm",
          "bill_length_mm",
          "bill_depth_mm",
          "body_mass_g"
        )
```

```r
      )
    ),
    mainPanel(
      plotOutput("scatter"),
      DTOutput("table")
    )
  )
)

server <- function(input, output, session) {
  filtered_data <- reactive({
    if (input$species == "All") {
      penguins
    } else {
      penguins[
        penguins$species == input$species,
      ]
    }
  })

  output$scatter <- renderPlot({
    ggplot(
      filtered_data(),
      aes(
        x = .data[[input$x_var]],
        y = .data[[input$y_var]],
        colour = species
      )
    ) +
      geom_point(alpha = 0.7, size = 3) +
      theme_minimal() +
      labs(
        x = input$x_var,
        y = input$y_var,
        colour = "Species"
      )
  })

  output$table <- renderDT({
    filtered_data()
  })
}

shinyApp(ui, server)
```

The app runs flawlessly on your machine.

4.2 Sending the App to Joe

You zip the directory and email it to Joe. Joe extracts the archive, opens a terminal, and attempts to run the app.

4.3 Error 1: Shiny Not Installed

```
Rscript -e "shiny::runApp('app.R')"
```

`there is no package called 'shiny'`. Joe installs it, which triggers compilation of `httpuv` and its system dependencies.

4.4 Error 2: System Libraries Missing

```
R -e "install.packages('shiny')"
```

The `httpuv` package fails to compile because `libssl-dev` and `libuv1-dev` are not present. Joe installs system libraries:

```
sudo apt install libssl-dev \
  libcurl4-openssl-dev libuv1-dev
```

4.5 Error 3: Package DT Not Found

After installing Shiny, the app fails because `DT` is not installed. Joe installs it, which requires additional compilation.

4.6 Error 4: palmerpenguins Not Found

The data package is missing. Joe installs it.

4.7 Error 5: Port Already in Use

Joe finally has all packages but gets `Address already in use` because another service occupies port 3838. He must figure out how to change the port or kill the conflicting process.

4.8 Error 6: Version Incompatibility

Joe's R version is 4.1; the app uses `.data[[]]` syntax from `rlang` 1.0+, which requires a newer `ggplot2` than what installed against R 4.1. The plot renders with an error.

After six errors and several hours, Joe has a running application. Every error was environmental.

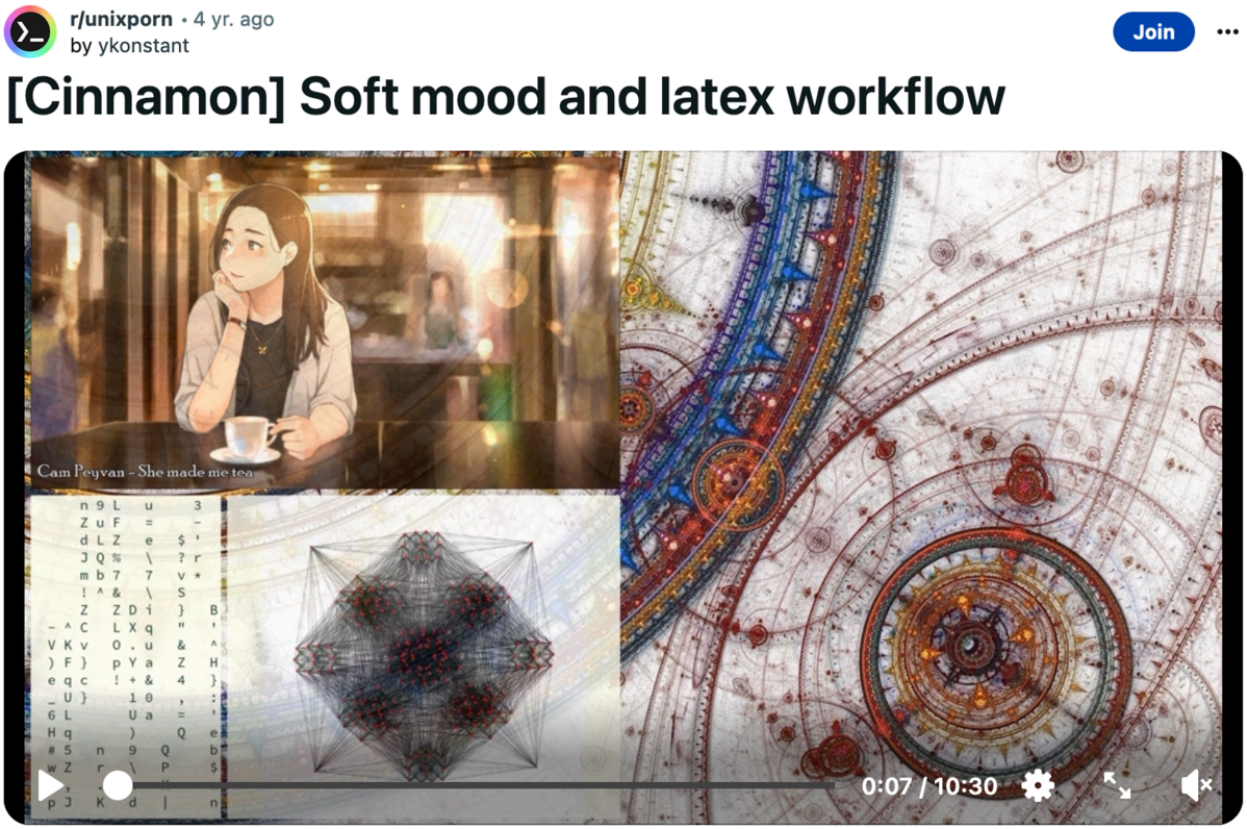

Figure 2: Hands assembling components into a container, representing the process of packaging an application for portability.

Packaging everything together so no piece is left behind.

5 The Docker Approach

The Docker approach eliminates every error by packaging the entire Shiny runtime into a single image.

5.1 The Dockerfile

The following Dockerfile starts from `rocker/shiny`, which includes R and Shiny Server pre-installed. It adds the required R packages and copies the application code into the image.

```dockerfile
FROM rocker/shiny:4

RUN apt-get update && apt-get install -y \
  libssl-dev \
  libcurl4-openssl-dev \
  libuv1-dev \
  && rm -rf /var/lib/apt/lists/*

RUN R -e "install.packages(c( \
  'ggplot2', \
  'DT', \
  'palmerpenguins', \
  'rlang' \
  ), repos = 'https://cran.r-project.org')"

RUN rm -rf /srv/shiny-server/*

COPY app.R /srv/shiny-server/app.R

EXPOSE 3838

CMD ["/usr/bin/shiny-server"]
```

Key decisions in this Dockerfile:

- **`FROM rocker/shiny:4`** provides R 4.x with Shiny Server pre-installed, eliminating the need to configure the web server manually.
- **System libraries** are installed before R packages to ensure compilation succeeds.
- **`rm -rf /srv/shiny-server/*`** removes the default sample apps, leaving only your application.
- **`COPY app.R`** places the application in the Shiny Server directory.
- **`EXPOSE 3838`** documents the port that Shiny Server listens on.
- **`CMD`** starts Shiny Server when the container launches.

5.2 Building the Image

Build the image with a tag:

```bash
docker build -t rgt47/penguins-shiny \
  --platform=linux/amd64 .
```

5.3 Sharing the Image

Option 1: Docker Hub

```bash
docker push rgt47/penguins-shiny
```

Joe pulls the image:

```
docker pull rgt47/penguins-shiny
```

Option 2: File transfer

```
docker save rgt47/penguins-shiny \
  | gzip > penguins-shiny.tgz
```

Joe loads the image:

```
docker load -i penguins-shiny.tgz
```

5.4 Running the Container

Joe runs the container with port mapping:

```
docker run -d --rm \
  -p 3838:3838 \
  --name penguins-app \
  rgt47/penguins-shiny
```

He opens a browser to `http://localhost:3838` and the application is running. No errors. No missing packages. No port conflicts (the `-p` flag maps the container port to any available host port).

To stop the application:

```
docker stop penguins-app
```

5.5 Alternative: Single-File Approach

For simpler applications without Shiny Server, you can run the app directly with `Rscript`:

```
FROM rocker/r-ver:4

RUN R -e "install.packages(c( \
  'shiny', \
  'ggplot2', \
  'DT', \
  'palmerpenguins' \
  ), repos = 'https://cran.r-project.org')"

COPY app.R /app/app.R

EXPOSE 3838
```

```
CMD ["Rscript", "-e", \
  "shiny::runApp('/app/app.R', \
  host='0.0.0.0', port=3838)"]
```

This approach uses `rocker/r-ver` (smaller base image) and runs the app directly without Shiny Server. The trade-off is the loss of Shiny Server features like logging and process management, but for single-app containers it is often sufficient.

5.6 Things to Watch Out For

1. **The `host` parameter must be `0.0.0.0`.** By default, `shiny::runApp()` binds to `127.0.0.1`, which is only accessible inside the container. Setting `host='0.0.0.0'` makes it reachable from the host machine.
2. **Port mapping requires the `-p` flag.** The container's port 3838 is not accessible from the host unless you explicitly map it with `-p 3838:3838` (or `-p 8080:3838` to use a different host port).
3. **Platform flag matters for Apple Silicon.** If you build on an M1/M2/M3 Mac, use `--platform=linux/amd64` to ensure the image runs on Intel-based Linux machines.
4. **Image size can be large.** The `rocker/shiny` base image is approximately 1.5 GB. Adding packages increases this. Consider `rocker/r-ver` with the single-file approach for smaller images.
5. **Shiny Server logs are inside the container.** Use `docker logs penguins-app` to view server output, or mount `/var/log/shiny-server` to a host directory for persistent logging.

Figure 3: A quiet desk with a closed laptop and a cup of tea, representing the conclusion of a completed workflow.

When the application launches on the first try, on any machine, the work is done.

5.7 Lessons Learnt

5.7.1 Conceptual Understanding

- Shiny applications compound the reproducibility problem because they require a running web server in addition to the standard R package ecosystem.
- Docker eliminates environment divergence by packaging the complete runtime – R, packages, system libraries, Shiny Server, and application code – into a single image.
- The `rocker/shiny` image provides a production-ready Shiny Server, while `rocker/r-ver` with `shiny::runApp()` offers a lighter alternative for single-app containers.
- Port mapping (`-p`) is the mechanism that connects the container's internal network to the host machine's browser.

5.7.2 Technical Skills

- Building a Dockerfile for Shiny requires installing system libraries (`apt-get install`) before R packages to ensure compilation of binary dependencies like `httpuv`.

- The `host='0.0.0.0'` parameter in `shiny::runApp()` is essential for making the application accessible from outside the container.
- `docker run -d` starts the container in detached mode, which is appropriate for long-running server applications.
- `docker logs` provides access to Shiny Server output for debugging runtime errors.

5.7.3 Gotchas and Pitfalls

- Forgetting to set `host='0.0.0.0'` is the most common reason a Dockerized Shiny app appears to start but shows a blank page in the browser.
- The default Shiny Server configuration serves all applications in `/srv/shiny-server/`; failing to remove sample apps results in a directory listing instead of your application.
- R package versions are not pinned in this Dockerfile; use `renv` for strict version control in production deployments.
- Large Docker images take significant time to push and pull; use `.dockerignore` to exclude unnecessary files from the build context.

5.8 Limitations

- Docker Desktop is required on the recipient's machine, which may not be permitted in all organisational environments.
- The `rocker/shiny` base image is approximately 1.5 GB, and the final image can exceed 2 GB, making it impractical for low-bandwidth sharing.
- R package versions are not pinned; future builds may install different versions, breaking reproducibility over time.
- This approach does not address authentication or access control; the Shiny app is accessible to anyone who can reach the port.
- Shiny Server in the free version does not support multiple concurrent users efficiently; for production deployment, consider ShinyProxy or Shiny Server Pro.
- The Dockerfile does not include a health check; orchestration tools (Kubernetes, Docker Compose) cannot verify the app is actually responding.

5.9 Opportunities for Improvement

1. Integrate `renv` into the Dockerfile to pin exact package versions and ensure long-term reproducibility.
2. Add a `docker-compose.yml` file for applications that require additional services (e.g., a PostgreSQL database for persistent storage).
3. Implement ShinyProxy for multi-user deployments with authentication and per-user container isolation.
4. Add a health check endpoint to the Dockerfile (`HEALTHCHECK CMD curl -f http://localhost:3838 || exit 1`) for container orchestration.
5. Use multi-stage builds to separate the package compilation environment from the runtime environment, reducing the final image size.
6. Explore GitHub Actions to automatically build and push the Docker image whenever the application code is updated.

6 Wrapping Up

The contrast between the naive approach and the Docker approach for Shiny applications is even more stark than for R Markdown. A static Rmd analysis might eventually render after fixing package issues. A Shiny app with a missing dependency produces a blank page and a cryptic log message that gives little indication of what went wrong.

The naive approach produced six separate errors, each requiring Joe to diagnose and fix an environment issue. The Docker approach produced zero errors. Joe pulled the image, ran a single command, and opened his browser to a working application.

What I learnt most from this experience is that interactive applications have a wider dependency surface than static analyses. The web server, the network binding, the port configuration, and the reactive dependencies all need to be correct simultaneously. Docker addresses all of these by fixing the entire stack at build time.

Main takeaways:

- The naive approach to sharing Shiny apps produced six distinct environment errors before the application could run.
- Docker eliminates all six errors by packaging the complete Shiny runtime into a single image.
- The `rocker/shiny` base image provides Shiny Server pre-configured; `rocker/r-ver` with `shiny::runApp()` offers a lighter alternative.
- Port mapping (`-p 3838:3838`) and host binding (`host='0.0.0.0'`) are the two settings most commonly missed when Dockerizing Shiny apps.

7 See Also

Related posts:

- "Sharing R Markdown Code Reproducibly with Docker" (the R Markdown equivalent of this workflow)
- "Configure the Command Line for Data Science Development" (terminal and Docker setup)

Key resources:

- Rocker Project: Docker containers for R
- rocker/shiny Docker image
- Shiny Server documentation
- ShinyProxy: open-source Shiny deployment
- Docker documentation
- renv: Project Environments for R

8 Reproducibility

This workflow was developed on macOS with Docker Desktop 4.x, R 4.4, and Shiny 1.8. The following files constitute the complete Docker-based Shiny application:

File	Purpose
`app.R`	Shiny application code
`Dockerfile`	Environment definition

To reproduce the complete workflow:

```
# Build the Docker image
docker build -t rgt47/penguins-shiny \
  --platform=linux/amd64 .

# Run the container
docker run -d --rm \
  -p 3838:3838 \
  --name penguins-app \
  rgt47/penguins-shiny

# Open browser to http://localhost:3838

# Stop the container
docker stop penguins-app
```

9 Let's Connect

- **GitHub:** rgt47
- **Twitter/X:** @rgt47
- **LinkedIn:** Ronald Glenn Thomas
- **Email:** rgtlab.org/contact

I would enjoy hearing from you if:

- You spot an error or a better approach to any of the code in this post.
- You have suggestions for topics you would like to see covered.
- You want to discuss R programming, data science, or reproducible research.
- You have questions about anything in this tutorial.
- You just want to say hello and connect.